

SystemVerilog Unified Revision Guide: Data Types, OOP, and Testbench Architecture

A Comprehensive Engineering Handbook and Dual-Track Revision Synthesis

Source Mapping: *First Revision (GitHub Lessons 01--21) & Second Revision (Laboratory Practice C1--C9, OOP Blocks, and Constrained Randomization)*

Scope: *SystemVerilog 2-State/4-State Types, Simulation Scheduler, Packed/Unpacked Arrays, Dynamic Arrays, Queues, Subroutines, Tasks vs Functions, Argument Passing Semantics (input, output, inout, ref), Automatic Storage, Classes, Handles vs Objects, Composition, Encapsulation, Copying Mechanisms (Handle Copy, Shallow Copy, Recursive Deep Copy), Inheritance, Dynamic Polymorphism with Virtual Methods, Constructor Chaining (super.new), and Constrained Randomization Fundamentals (rand, constraints, assert(randomize()), lifecycle).*

Language Standard: *IEEE Std 1800-2017 (SystemVerilog LRM).*

Simulator Verification: *AMD Vivado Simulator 2024.1 (64-bit build 5076996) -- 32/32 Deterministic Checks Passed.*

Contents

Executive Quick-Recall Principles	2
Master Cross-Revision Question & Knowledge Guide	3
Part I: Data Types, Representation & Simulation Foundations	14
Part II: Subroutines & Argument Passing Semantics	19
Part III: Object-Oriented Programming (OOP) in SystemVerilog	21
Part IV: Practice Code Bug Audit and Code Repairs	25
Part V: Active Recall Self-Test & Detailed Solutions	27
Part VI: Simulation Verification Report & Standards Appendix	28

Executive Quick-Recall Principles

Instead of condensing complex engineering concepts into diluted tabular rows, the twelve core tenets of SystemVerilog verification foundations are presented below with their strict IEEE Std 1800-2017 language semantics and practical verification consequences.

Principle 1: 2-State vs 4-State Default Initialization

In SystemVerilog, 2-state types (`bit`, `byte`, `shortint`, `int`, `longint`) initialize deterministically to 0 at time zero. In contrast, 4-state integral types (`logic`, `reg`, `integer`, `time`) initialize strictly to X (unknown), never Z. In verification, uninitialized 4-state signals immediately propagate X throughout downstream combinational and sequential logic, exposing uninitialized registers or unasserted reset trees. Conversely, 2-state variables default to zero, which can mask hardware initialization bugs if used erroneously to model hardware reset lines.

Principle 2: Signedness and Arithmetic Weighting Rules

The types `byte`, `shortint`, `int`, `longint`, and `integer` are **signed by default** (interpreted as two's complement). The types `bit` and `logic` are **unsigned by default**. Width and signedness are completely orthogonal: an 8-bit byte `b = 130` stores 8'h82 (8'b1000_0010), where bit 7 represents the sign weight -128 , evaluating to $-128 + 2 = -126$. Assigning the identical value to an unsigned vector `bit [7:0] u = 130` evaluates to $+128 + 2 = +130$. Signedness determines sign-extension during width expansion and affects relational comparisons (`<`, `>`, `<=`, `>=`).

Principle 3: Unpacked Concatenation vs Assignment Patterns

SystemVerilog strictly differentiates aggregate assignments. An unpacked array concatenation `{1, 0, 1, 1}` is legal syntax when the target of the assignment is an unpacked array or queue. However, structured array initialization generally uses the assignment pattern operator with a leading apostrophe: `'{1, 0, 1, 1}`. Pattern replication syntax requires the apostrophe multiplier `'{N{val}}`, and fallback filling requires the pattern key `'{default: val}`. Mixing these rules causes tool syntax errors.

Principle 4: Array Size Querying and Rejection of `$sizeof()`

SystemVerilog provides three dedicated inquiry functions for array dimensions: `$size(arr)` returns the element count of the specified dimension; `$bits(arr)` returns the total number of bits required to serialize the entire object; and the `.size()` built-in method returns the dynamic element count of dynamic arrays and queues. The operator `$sizeof()` is a C/C++ construct that does NOT exist in SystemVerilog. Attempting to call `$sizeof()` triggers an immediate compilation failure.

Principle 5: The Zero-Simulation-Time Law of Procedural Loops

All procedural loop constructs (`for`, `foreach`, `repeat`, `while`, `do...while`) execute instantaneously within the simulator's active event region. Iterating a loop 1,000 or 1,000,000 times consumes exactly **zero simulation time** (`$0\text{ns}`). Simulation time advances ONLY when an explicit blocking timing control (`#delay`, `@(posedge clk)`, `wait`) is reached inside the procedural thread. Loops without timing controls execute as combinational bursts.

Principle 6: Subroutines: Tasks vs Functions Time Boundary

The absolute boundary dividing a task from a function is **simulation time consumption**. A function cannot consume simulation time under any circumstances: it cannot contain delay controls (`#`), event controls (`@`), `wait` statements, or calls to blocking tasks. A task is permitted to suspend execution and advance simulation time. Procedural functions in SystemVerilog fully support output, inout, and ref arguments -- the widespread belief that functions cannot have output arguments is an outdated Verilog-1995 misconception.

Principle 7: Argument Passing: Stack Copying vs Direct Memory Aliasing

The input mode passes arguments by value, copying data onto the subroutine's local stack frame upon entry; any local modifications operate exclusively on the local copy, perfectly isolating the caller. The ref mode passes a direct memory reference, aliasing the caller's actual variable so that modifications are reflected in the caller immediately. Passing large dynamic arrays or transaction objects by ref eliminates costly memory allocation and copying overhead. In static scopes (modules and interfaces), subroutines using ref arguments MUST be declared with the `automatic` lifetime keyword.

Principle 8: Passing Class Handles: Value of Handle vs Mutating Object State

Passing a class handle as an input argument passes the handle (pointer) by value. Both caller and callee handles refer to the exact same object in heap memory. Consequently, modifying an internal property (`pkt.payload = 8'hAA`) mutates the caller's object directly. However, reassigning the handle itself (`pkt = null` or `pkt = new()`) rebinds only the callee's local stack pointer; upon return, the caller's handle remains valid and points to the mutated heap object.

Principle 9: Object Copying Taxonomy: Handle Copy vs Shallow Copy vs Deep Copy

Handle assignment (`h2 = h1`) merely creates a second pointer pointing to the same existing heap object (one object, two aliases). Built-in shallow copy (`h2 = new h1`) allocates a new outer object and copies scalar fields by value, but copies nested object handles by reference, meaning both outer objects share the identical nested child object. Shallow copy never invokes the class constructor `new()` or property initializers. Custom deep copy recursively calls copy methods on all nested objects, guaranteeing complete heap memory isolation.

Principle 10: Dynamic Polymorphism and Virtual Method Dispatch

Declaring a method `virtual` in a base class informs the simulator's compiler to use dynamic runtime dispatch rather than static compile-time binding. When a child class handle is upcast into a base class handle (`Base b = child_inst;`) and `b.display()` is called, the simulator inspects the actual object type residing on the heap (which is `Child`) and executes `Child.display()`. If the base class method is not declared `virtual`, static dispatch forces the execution of `Base.display()`.

Principle 11: Constructor Chaining and the `super.new` Placement Law

In derived class constructors, calling `super.new(...)` invokes the parent class constructor to ensure that inherited base properties are fully initialized before derived properties are configured. SystemVerilog mandates that `super.new(...)` must be the **first executable statement** inside the derived constructor. If omitted, the compiler inserts an implicit parameterless `super.new()`, which fails compilation if the parent constructor requires mandatory arguments without defaults.

Principle 12: Constrained Randomization Rigor and Verification Error Traps

Randomization solver failures occur when constraints are contradictory, when solver bounds are over-constrained, when solver timeouts occur, or when calling `randomize()` on a `null` handle. In SystemVerilog testbenches, calling `randomize()` without verifying its return value is a critical verification bug because stale or unrandomized stimulus silently propagates into the DUT. Verification best practice mandates validating every call using an immediate assertion: `assert(g.randomize()) else $fatal(...)`. Furthermore, inside stimulus loops, allocating fresh transaction instances (`g = new();`) is mandatory to avoid handle overwriting in downstream FIFOs and mailboxes.

Master Cross-Revision Question & Knowledge Guide

This guide synthesizes every question, discussion topic, and code exercise from both revision tracks:

- **First Revision Track:** GitHub repository lessons `SV_Basics/Codes/01 to 21` and `QUESTION_TO_CODE_INDEX.md`.
- **Second Revision Track:** User laboratory practice snippets `C1 through C9`, OOP code blocks, and Constrained Randomization practice.

Domain 1: Scalar Data Types, Signedness, and Value Systems

Questions Addressed from the Repository and Practice

1. From Lesson 04 (README Question 1): What is the initial value of a 4-state variable?
2. From Lesson 04 (README Question 2): Why can a wire not be used in a procedural block assignment?
3. From Lesson 04 (README Question 3): Why is `reg` not allowed at the output of a continuously driven gate or module?
4. From Practice C1 (Line 4): Is `bit` a 2-state signed variable type?
5. From Practice C2 (Line 27): Why is `byte var = -126;` called a variable, and why is `var` illegal as an identifier?

Exhaustive Technical Explanation

A **variable** in SystemVerilog is a data object that holds and retains its assigned value across simulation time until a subsequent procedural assignment actively overwrites it. A **net** (such as `wire`) does not store state; it models physical hardware connectivity and continuously resolves the driven state from all connected drivers. In procedural blocks (`initial` and `always`), target objects must be variables (`logic`, `reg`, `bit`, `byte`, `int`) because procedural statements represent sequential execution that updates stored state. Continuous assignments (`assign`) drive nets or 4-state variables.

The initial value of an uninitialized 4-state variable (`logic`, `reg`, `integer`, `time`) is strictly `X` (**unknown**). It is never `Z`! High-impedance `Z` represents an un-driven tri-state condition on a physical wire; it never serves as the default state of storage variables.

The data type `bit` is a 2-state type defaulting to `0`. Crucially, `bit` is **unsigned by default**. A single `bit` variable holds `0` or `1`. To create a signed `bit` vector, the designer must explicitly write `bit signed [7:0] s_byte;`. In contrast, `byte` is an 8-bit integral type that is **signed by default** under two's complement arithmetic, with a legal range of `-$128$` to `+$127$`. When storing `8'h82` (`8'b1000_0010`), a

byte evaluates to -126 because the most significant bit carries sign weight -128 ($-128 + 2 = -126$). A bit `[7:0]` storing `8'h82` evaluates to $+130$ because bit 7 carries unsigned weight $+128$.

Finally, the identifier `var` in `byte var = -126;` is a reserved SystemVerilog language keyword (used in explicit variable declarations such as `var logic [7:0] bus;`). Using `var` as a variable identifier triggers a compile-time syntax error.

"How to Speak It" Interview Script

"In SystemVerilog, width and signedness are orthogonal architectural properties. The byte type is an 8-bit signed two's complement integer ranging from -128 to 127, whereas bit [7:0] is an 8-bit unsigned vector ranging from 0 to 255. Even though both store the exact same hexadecimal value 8'h82, their arithmetic interpretations diverge completely: the MSB in a byte carries a negative weight of -128, yielding -126, whereas in a bit vector it carries +128, yielding 130. Uninitialized 4-state variables always default to X, which provides essential uninitialized-state detection in RTL, while 2-state variables default to 0."

Domain 2: Simulation Time, Timescale, Scheduling, and Clock Generation

Questions Addressed from the Repository and Practice

1. From Lesson 01 (README Questions 1 & 2): What does the time-zero comment mean, and what value does a variable retain through simulation?
2. From Lesson 02 (README Questions 1, 2 & 3): Why does a testbench always block not need a sensitivity list, why does a design always block need one, and why must clock variables be initialized?
3. From Lesson 02 (README Math Audit): What is the exact period and frequency of a clock toggling `#20 clk = 1; #10 clk = 0;`?
4. From Lesson 03 (README Analysis): What is the difference between a time-zero startup offset and a true steady-state phase shift?
5. From Practice C2 (Lines 29-30): What is the difference between time and realtime, and between `$time` and `$realtime`?

Exhaustive Technical Explanation

Simulation time in SystemVerilog is established by the ``timescale <unit> / <precision>`` directive. The `**time unit**` defines the physical measurement of unsuffixed delays (e.g., `#10` under `1ns/1ps` is `$10\text{ns}`). The **time precision** defines the internal scheduler's rounding quantum; all scheduled delays are rounded to the nearest multiple of the precision before entry into the event wheel.

The system function `$time` returns the current simulation time as a 64-bit integer scaled to the current time unit. The function `$realtime` returns simulation time as a 64-bit real floating-point number, preserving fractional delay precision. Variables declared as `time` or `realtime` are static storage locations; they do not automatically advance or track time -- they store only values explicitly assigned to them.

In testbench clock generation, an `always` block without a sensitivity list executes continuously as an infinite procedural loop driven by internal delay controls (`#half_period clk = ~clk;`). In synthesis RTL modeling, combinational `always @(*)` or clocked `always @(posedge clk)` blocks require sensitivity lists to instruct the simulator which net or variable transitions must trigger process evaluation. Clock variables in testbenches must be initialized at time zero (`bit clk = 0;` or `initial clk = 0;`), otherwise a 4-state clock initializes to X, and toggling `~clk` results in `~X = X`, stalling clock generation forever.

In GitHub Lesson 02, a clock labeled `clk25Mhz` toggled `#20 clk = 1; #10 clk = 0;`. The total period was `$20\text{ns} + 10\text{ns} = 30\text{ns}`. The resulting frequency was `f = 1000 / 30 = 33.33\text{MHz}`, NOT 25 MHz! For an authentic 25 MHz clock, period `T = 1000 / 25 = 40\text{ns}`, requiring 20 ns high and 20 ns low.

A time-zero startup offset (`initial begin #10 clk = 0; forever #5 clk = ~clk; end`) delays the inception of toggling by 10 ns from time zero. A true steady-state phase shift represents the ongoing time or angular delay between corresponding edges of two active synchronous clocks running at identical or related frequencies.

"How to Speak It" Interview Script

"Simulation precision is not merely a formatting switch; it is the fundamental scheduling quantum of the simulation kernel. When generating clocks in testbenches, the period equation is $T_{ns} = 1000 / \text{Frequency_MHz}$. For a 25 MHz clock with 50% duty cycle, the period is 40 ns, requiring 20 ns high and 20 ns low. Clock variables must always be initialized to 0 or 1 at time zero because toggling an uninitialized 4-state X produces X indefinitely, creating a deadlocked simulation thread."

Domain 3: Array Shapes, Concatenations, and Sizing Functions

Questions Addressed from the Repository and Practice

1. From Lesson 05 (README Questions 1 & 2): How does `bit arr[] = {1,0,1,1};` get a size of four, and can an uninitialized dynamic array accept an element assignment?

2. From Lesson 07 (README Question 4): Why does `$sizeof(arr)` fail to compile in SystemVerilog?
3. From Practice C3 (Lines 44-46): What are the proper inquiry functions to query array size and bit length?

Exhaustive Technical Explanation

SystemVerilog categorizes arrays into **packed** and **unpacked** formats. A packed array is declared with dimensions before the identifier name (`bit [3:0][7:0] p_word;`); it is guaranteed to be stored as a single, contiguous bit vector in simulator memory, supporting arithmetic, bitwise logic, and slicing. An unpacked array is declared with dimensions after the identifier name (`bit u_mem [4][8];` or `int arr[];`); each element occupies independent simulator memory.

When declaring a dynamic array with an immediate unpacked concatenation, such as `bit arr[] = {1, 0, 1, 1};`, the compiler infers the dimension from the number of elements in the right-hand concatenation list, allocating exactly 4 elements at indices 0, 1, 2, and 3. However, an uninitialized dynamic array declared without an initial size (`bit arr3[];`) has an initial size of 0. Attempting to assign an element directly (`arr3[0] = 1;`) triggers an out-of-bounds fatal runtime error. The array must first be allocated memory using the dynamic array new-constructor: `arr3 = new[4];`.

SystemVerilog does not include C's `$sizeof` operator. To inspect arrays, SystemVerilog provides:

- `$size(arr)`: Returns the total number of elements in the specified dimension.
- `$bits(arr)`: Returns the total number of bits required to serialize the entire object.
- `arr.size()`: Built-in method returning the element count of dynamic arrays and queues.
- `$dimensions(arr)`: Returns the total number of dimensions (packed plus unpacked).

"How to Speak It" Interview Script

"In SystemVerilog, memory layout depends on dimension placement: dimensions before the identifier define contiguous packed bit-vectors suitable for bit-level arithmetic, whereas dimensions after the identifier define unpacked arrays of isolated memory elements. Dynamic arrays require runtime heap allocation before index access. When querying dimensions, always use `$size` for element count, `$bits` for serialized bit-width, and `.size()` for dynamic structures -- never use C's `$sizeof`, which is not recognized by the LRM."

Domain 4: Array Initialization Patterns and Repetition

Questions Addressed from the Repository and Practice

1. From Lesson 05 (README Question 3): What does the initialization comment distinguish between aggregate assignment formats?
2. From Practice C4 (Lines 58-64): What is the syntax for positional assignment, replication, and default keys?

Exhaustive Technical Explanation

SystemVerilog provides two distinct aggregate initialization constructs:

1. **Unpacked Array Concatenation** (`{...}`): Allowed when assigned to an unpacked array or queue target. For instance, `bit arr[] = {1, 0, 1, 1};` assigns four elements directly without requiring an apostrophe.
2. **Assignment Patterns** (`'{...}`): The formal, strongly-typed SystemVerilog mechanism for initializing arrays and structures. It requires an apostrophe prefix.

Within an assignment pattern, SystemVerilog supports multiple initialization idioms:

- **Positional Assignment**: `int a[5] = '{1, 3, 4, 2, 2};` binds values to indices \$0, 1, 2, 3, 4\$ in order.
- **Pattern Replication**: `int b[5] = '{5{0}};` creates 5 replicated instances of 0. Note that this requires the apostrophe before the outer brace: `'{ N { val } }.`
- **Default Key Assignment**: `int c[5] = '{default: 4};` fills every element of the array with value 4.
- **Combined Positional and Default**: `int d[9] = '{1, 2, default: 0};` assigns indices 0 and 1 explicitly and initializes indices 2 through 8 to 0.

"How to Speak It" Interview Script

"SystemVerilog assignment patterns use the apostrophe-brace syntax '{...}' to establish unambiguous aggregate types. They provide tremendous flexibility in verification testbenches: we can assign positional values, replicate elements using '{N{val}}', or assign default fallbacks using '{default: value}'. This guarantees that complex multidimensional arrays and scoreboard lookup tables can be initialized robustly without manual loops."

Domain 5: Array Iteration, Loop Mechanics, and the Zero-Time Law

Questions Addressed from the Repository and Practice

1. From Lesson 06 (README Question 1): Why does foreach visit indices zero through nine?
2. From Lesson 10 (README Question 5): How much simulation time do loops consume?
3. From Practice C5 (Lines 70-98): How do for, foreach, and repeat compare in stimulus generation?

Exhaustive Technical Explanation

The `foreach(arr[i])` loop is a specialized array traversal construct designed specifically for SystemVerilog unpacked arrays, dynamic arrays, and queues:

1. The index variable `i` is **automatically declared** with local scope strictly within the loop body.
2. The index variable is **read-only**; attempting to procedurally assign `i = 5`; inside the loop causes a compilation error.
3. The simulator inspects the declared bounds of the array. If an array is declared `int a[1:5]`;, `foreach` visits \$1, 2, 3, 4, 5\$. If declared `int a[5]`; (shorthand for `[0:4]`), it visits \$0, 1, 2, 3, 4\$. For multidimensional arrays, nested indexing is written concisely: `foreach(matrix[r, c])`.

The loop constructs `for`, `repeat(N)`, and `while` require explicit variable declarations. In Practice C5, writing `repeat(10) begin arr[i] = i; i++; end` resulted in a compile failure because `i` had not been declared in that process scope.

The Golden Law of Simulation Time: Iteration alone consumes **ZERO simulation time**! Executing a loop of 1,000, 10,000, or 1,000,000 iterations takes 0 nanoseconds on the simulation timeline unless an explicit timing control (`#delay`, `@(posedge clk)`, `wait`) is encountered inside the loop body.

"How to Speak It" Interview Script

"The foreach loop provides bounds-safe, self-indexing iteration where loop variables are implicitly declared and read-only. Crucially, procedural loops in SystemVerilog take zero simulation time -- they execute entirely within an active evaluation iteration at the current simulation timestamp. To model physical hardware duration, explicit timing controls or clock event synchronizations must be inserted within the loop body."

Domain 6: Array Copying and Whole-Array Equality

Questions Addressed from the Repository and Practice

1. From Lesson 07 (README Questions 1 & 2): Why is copying used in a scoreboard, and should whole-array comparison return true?
2. From Practice C6 (Lines 105-117): How does whole-array assignment execute, and what is the difference between `==` and `===`?

Exhaustive Technical Explanation

When two fixed unpacked arrays of identical type and size are assigned (`arr2 = arr1`);, SystemVerilog performs an immediate, element-by-element **deep value copy**. Modifying elements of `arr2` subsequent to the assignment has absolutely zero effect on `arr1`. This is widely used in scoreboard components: when a transaction packet arrives from a monitor, its payload array is copied into an expected queue or scoreboard storage array so that downstream driver activity cannot mutate the reference data.

Whole-array comparison evaluates equality across all elements:

- **Logical Equality** (`arr3 == arr4`): Compares all elements pairwise. If all pairs match, it returns `1'b1`. If any pair differs, it returns `1'b0`. If any element contains `x` or `z` and no definite mismatch exists, it returns `1'bx`.
- **Case Equality** (`arr3 === arr4`): Treats `x` and `z` as literal values. It returns a deterministic Boolean `1'b1` or `1'b0` regardless of unknown or high-impedance states.

"How to Speak It" Interview Script

"Whole-array assignment in SystemVerilog performs a complete value copy across all elements, ensuring caller and receiver data structures remain completely decoupled. For comparison, logical equality `==` returns X if any compared bit is unknown, whereas case equality `===` performs exact literal state matching across 0, 1, X, and Z, which is crucial when verifying reset or uninitialized bus states in scoreboards."

Domain 7: Dynamic Array Allocation, Resizing, and Memory Management

Questions Addressed from the Repository and Practice

1. From Lesson 07 (README Questions 3 & 4): Why does deleting a dynamic array not print `xxxx`, and why is `new` needed to add elements?

2. From Practice C7 (Lines 123-138): How does dynamic array resizing preserve existing data, and what is the rule for assigning dynamic arrays to fixed arrays?

Exhaustive Technical Explanation

A dynamic array is declared with empty brackets (`int arr[];`). At declaration time, it has a size of 0 and points to no memory on the simulation heap. Memory is allocated dynamically at runtime using the dynamic-array new-constructor: `arr = new[5];`.

When `arr.delete()` is called, the simulator reclaims the heap memory and resets the array size to 0. Printing an array whose size is 0 (`$display("%0p", arr)`) outputs an empty list '{}'. It does NOT print xxxx because there are no longer any elements in existence to hold an unknown state!

To resize a dynamic array while preserving its current contents, the old array handle is passed as an argument to the new-constructor:

```
int arr[];
arr = new[5];           // Size 5
// ... populate arr ...
arr = new[30](arr);     // Resized to 30; first 5 elements preserved!
```

The first 5 elements retain their values, while the remaining 25 new elements are initialized to the type default (0 for int). If the new size is smaller than the original, the array is truncated to the first N elements.

When assigning a dynamic array into a fixed-size array (`fixed = arr;`), the dynamic array's current run-time size must **identically match** the declared size of the fixed array. If the sizes differ, the simulator terminates with a fatal runtime error.

"How to Speak It" Interview Script

"Dynamic arrays provide runtime flexibility when the collection size is unknown at compile time. Memory is allocated using `new[N]`, and existing data is preserved during resizing by passing the handle into the constructor: `newN_new`. Calling `delete()` frees the heap allocation and reduces the size to zero. When copying a dynamic array to a fixed array, the runtime sizes must match exactly, or a fatal elaboration/simulation error occurs."

Domain 8: Queue Syntax, Built-in Methods, and Performance

Questions Addressed from the Repository and Practice

1. From Lesson 08 (README Question 1): Where should data be declared for queue operations?
2. From Practice C8 (Lines 144-163): What is the syntax for queue concatenation, boundary pushes/pops, insertion, and deletion?

Exhaustive Technical Explanation

A **queue** is declared using the bounded or unbounded dollar syntax: `int q[$];`. Queues are indexed from 0 to N (where N represents the last valid index). Unlike dynamic arrays, queues do not require explicit `new[]` allocations; the simulator automatically manages memory blocks. Queues accept unpacked concatenations directly without an apostrophe: `q = {1, 3, 4};`.

Queues provide high-performance $O(1)$ operations at both ends:

- `push_front(val)`: Inserts `val` at index 0, shifting existing elements up by one index.
- `push_back(val)`: Appends `val` at the end of the queue (index N).
- `pop_front()`: Removes and returns the element at index 0.
- `pop_back()`: Removes and returns the element at index N (the last element).
- `insert(idx, val)`: Inserts `val` at index `idx`, shifting subsequent elements right. (Note: Practice C8 erroneously wrote `arr.insert(2)`, which caused a syntax error because `insert()` requires both index and value).
- `delete(idx)`: Removes the single element at index `idx`.
- `delete()`: Empties the entire queue (size becomes 0).

Because queues provide constant-time front and back insertions and deletions without requiring array-wide reallocations, they are universally preferred over dynamic arrays for modeling hardware FIFOs, mailbox buffers, and transaction scoreboard pipelines.

"How to Speak It" Interview Script

"Queues are the standard data structure for transaction pipelines in SystemVerilog. They provide constant-time $O(1)$ push and pop operations at both boundaries without manual memory management. We initialize them cleanly using unpacked concatenation, query them using `.size()`, and pop items using `pop_front()` or `pop_back()`. In practice, we always prefer queues over dynamic arrays whenever elements are frequently pushed and popped dynamically."

Domain 9: Subroutines: Tasks vs Functions

Questions Addressed from the Repository and Practice

1. From Lesson 10 (README Questions 1, 2, 3 & 4): Why can a function not contain a delay, and can a function receive existing input variables?
2. From Practice C9 (Lines 167-169): Can a SystemVerilog function have output ports?

Exhaustive Technical Explanation

Subroutines in SystemVerilog encapsulate procedural algorithms. The core language rules governing tasks and functions are:

1. **Simulation Time:** A **function** cannot consume simulation time. It is forbidden from containing `#delay`, `@event`, `wait`, or invoking blocking tasks. It must execute to completion within a single evaluation step. A **task** can consume simulation time, suspend execution, wait for clock edges, and call other tasks.
2. **Return Values:** A non-void function returns a single value directly via its name or an explicit `return` statement, allowing it to be called inside expressions (`if (is_valid(pkt))`). A task does not return an expression value and is invoked strictly as a procedural statement (`send_pkt(pkt);`).
3. **Refuting the Myth: "Functions Cannot Have Output Ports":** In Verilog-1995, functions were strictly restricted to input arguments. However, in modern SystemVerilog (IEEE Std 1800-2017 Clause 13.4), **procedural functions fully support** output, inout, and ref arguments! When called in procedural blocks (`initial`, `always`, or `tasks`), a function can legally return multiple values through output arguments. Functions with output arguments cannot be called inside continuous assignments or event expressions.

"How to Speak It" Interview Script

"The defining architectural distinction between tasks and functions is time consumption: functions are strictly zero-time routines that execute instantaneously, whereas tasks can advance simulation time using delays, clock-edge waits, and events. While tasks cannot be called within expressions, procedural functions can declare output and ref arguments, allowing zero-time subroutines to compute and return multiple values simultaneously."

Domain 10: Argument Passing Modes and Memory Aliasing

Questions Addressed from the Repository and Practice

1. From Lesson 11 (README Questions 1 & 2): Will an argument update inside a pass-by-value task reflect outside, and why discuss pass-by-value for scalars?
2. From Lesson 12 (README Question 1): Why is copying an array to the stack considered non-optimal?
3. From Practice C9 (Lines 255 & 275): How do input, output, inout, ref, and const ref operate at the memory level?

Exhaustive Technical Explanation

SystemVerilog supports five distinct argument passing mechanisms:

- **input (Pass-by-Value):** Default mode. When a subroutine is called, the simulator allocates a private storage slot on the subroutine's local stack frame and copies the caller's value into it. Any modifications made to the argument inside the subroutine alter only this local copy. The caller's variable remains completely unchanged.
- **output (Pass-by-Value Return):** The subroutine formal begins with the default value of its type. Modifications update the local formal. Upon subroutine exit, the final value of the formal is copied back into the caller's variable.
- **inout (Bidirectional Pass-by-Value):** Copies the caller's value in on entry and copies the final local formal value back out upon return.
- **ref (Pass-by-Reference):** Does NOT create a local copy. Instead, it passes a direct reference (memory alias) to the caller's actual variable. Any modification made to a ref formal is instantly reflected in the caller's variable in real time, without waiting for the subroutine to exit.
- **const ref (Read-Only Reference):** Passes a direct memory alias for high performance, but the compiler enforces that the subroutine cannot modify the variable. Attempting to assign to a const ref formal triggers a compilation error.

Passing large unpacked arrays (such as a 1024-byte packet buffer) by value (`input`) forces the simulator to copy every element onto the call stack for every invocation, degrading simulation speed and consuming excessive memory. Passing large arrays via `ref` or `const ref` passes only a pointer reference, delivering maximum simulation performance while protecting data integrity when using `const ref`.

"How to Speak It" Interview Script

"Pass-by-value is the default mechanism: it creates a private stack copy upon entry, isolating the caller's variable from internal changes. Pass-by-reference, specified via the ref keyword, creates an immediate memory alias to the caller's variable. In verification testbenches, pass-by-reference is essential for two reasons: first, performance optimization, by avoiding costly stack-copying of large payload arrays; and second, real-time synchronization, allowing concurrent processes to observe variable mutations immediately. When caller modification is not intended, we use const ref to achieve reference performance with compiler-enforced read-only safety."

Domain 11: Storage Lifetimes: Static vs Automatic Scopes

Questions Addressed from the Repository and Practice

1. From Lesson 11 (README Question 1): Why is the automatic keyword required?
2. From Practice C9 (Lines 277-280): Why does task automatic swap(ref bit [1:0] a, b) require automatic?

Exhaustive Technical Explanation

In SystemVerilog, storage lifetime dictates how variables are allocated in memory:

- **static Lifetime:** Variables are allocated a single, fixed memory location at time zero. This storage location is shared across all calls to the subroutine. If a static task is invoked concurrently by multiple threads (such as inside a fork...join), the threads overwrite each other's local variables, causing severe race conditions. In modules, interfaces, and packages, subroutines default to static lifetime unless explicitly declared otherwise.
- **automatic Lifetime:** Variables are allocated dynamically on the call stack for each individual invocation. When the subroutine finishes, its stack frame is popped and memory is freed. Class methods default to automatic lifetime.

Strict IEEE Language Requirement (Clause 13.5.2): Subroutines declared inside a module or interface that accept ref arguments **MUST be declared with the automatic keyword!** In a static subroutine, formal arguments have fixed global storage locations. Creating an alias to a static formal violates memory safety when multiple calls occur. Therefore, writing task swap(ref bit [1:0] a, b); inside a module without automatic triggers an immediate compile-time error.

"How to Speak It" Interview Script

"In SystemVerilog, subroutines inside modules default to static lifetime, meaning their local variables share a single physical memory address across invocations. Marking a subroutine automatic forces the simulator to allocate a fresh, re-entrant call frame on the stack for every execution. Crucially, IEEE Std 1800-2017 mandates that any subroutine using ref arguments inside a module or interface must be declared automatic to ensure thread-safe pointer aliasing."

Domain 12: Classes, Handles, Objects, and Lifecycle

Questions Addressed from the Repository and Practice

1. From Lesson 09 (README Questions 1, 2, 3, 4, 5 & 6): Can a class handle access class members before construction? What does it mean that class objects are dynamic? Does the object need to be kept for the whole simulation? What does new() do?
2. From Practice C9 (Lines 175-185): What is the relationship between f1, f1 = new(), and f1 = null?

Exhaustive Technical Explanation

Object-oriented programming in SystemVerilog separates the **class type**, the **class handle**, and the **heap object**:

- **Class Type:** The compile-time blueprint defining properties and methods.
- **Class Handle:** A typed pointer variable allocated on the stack. At declaration (Packet pkt;), the handle contains the default value null. It points to no memory.
- **Heap Object:** A dynamic memory block allocated on the simulation heap when new() is executed.

Attempting to access properties or call non-static methods through an unconstructed handle (pkt.data = 5; when pkt == null) results in an immediate fatal **null-pointer dereference** runtime crash.

Class objects are dynamic because they are constructed and destroyed dynamically during simulation runtime, unlike static module instances which exist throughout the entire simulation run.

SystemVerilog features **automatic garbage collection**. When a designer executes pkt = null;, the handle is unlinked from the object. The object remains alive in heap memory as long as at least one other handle references it. When all handles referencing an object are reassigned or go out of scope, the simulator's garbage collector automatically reclaims the memory. There is no manual free() or delete operator for class objects in SystemVerilog.

"How to Speak It" Interview Script

"In SystemVerilog, a class variable is purely a handle -- a typed pointer initialized to null. Memory is allocated on the heap only when we call new(). Dereferencing a null handle triggers a fatal simulation crash. Unlike C++, SystemVerilog features automated garbage collection: when we assign a handle to null, we merely sever the reference. Once an object has zero active references, the simulation engine automatically reclaims its memory."

Domain 13: Constructors, this, and Member Scope

Questions Addressed from the Repository and Practice

1. From Lesson 13 (README Questions 1 & 2): Why can a constructor not have void as a return type, and what does the handle hold after construction?
2. From Lesson 14 (README Questions 1 & 3): Is class scope public by default, and why does a getter need a return type?
3. From Practice C9 (Lines 320-335): How is this used in constructor initialization?

Exhaustive Technical Explanation

A class constructor is declared as function `new(...)`.

1. **No Return Type:** A constructor cannot declare any return type -- not even void. Declaring function `void new();` causes a compilation error. The constructor implicitly allocates memory on the heap and returns the handle to the newly constructed object.
2. **Disambiguation with this:** When constructor argument names match class property names, the `this` keyword resolves scope ambiguity. The pointer `this` refers explicitly to the current class instance:

```
``systemverilog class Packet; int length; function new(int length); this.length = length; // 'this.length' is the
property; 'length' is the argument endfunction endclass ``
```

3. **Public by Default:** Unlike C++ where class members default to private, all properties and methods in SystemVerilog classes are **public by default**. They can be read and written directly from any scope using the handle and dot operator (`pkt.length = 100;`).
4. **Getters as Functions:** When encapsulation is applied using access modifiers, getters must be declared as non-void functions so that they can return values directly inside expressions (`if (pkt.get_length() > 64)`). A task cannot be used as an expression getter because tasks do not return values.

"How to Speak It" Interview Script

"A SystemVerilog constructor is declared as function new() with no return type specified, because it implicitly constructs the object and returns the object handle. All class members are public by default. We use the this keyword inside the constructor to disambiguate class properties from local arguments of the same name. Encapsulated properties are exposed via getter functions, which can be evaluated directly inside arithmetic or procedural expressions."

Domain 14: Encapsulation and Class Composition

Questions Addressed from the Repository and Practice

1. From Lesson 14 (README Questions 2, 4 & 5): What does `local` do to a member, can an initial block replace a constructor in a composed class, and why does a task not work as a getter?
2. From Practice C9 (Lines 345-375): How do `local`, `protected`, and composition interact?

Exhaustive Technical Explanation

Encapsulation safeguards internal class state using access qualifiers:

- `local`: Restricts access strictly to methods of the declaring class. Derived subclasses and external scopes cannot access local members.
- `protected`: Restricts access to the declaring class AND any derived subclasses (extends). External scopes cannot access protected members.

Class Composition (Has-A Relationship): Composition occurs when an outer class contains a class handle to an inner class as a property:

```
class Inner;
  int data = 10;
endclass

class Outer;
  Inner in_inst;
```

```
function new();
    in_inst = new(); // Outer constructor constructs inner instance
endfunction
endclass
```

Prohibition of initial Blocks: A class **cannot contain an initial block!** The initial block is a static procedural construct reserved exclusively for modules, interfaces, and programs. Placing an initial block inside a class definition triggers a compilation error. All object initialization must occur within the new() constructor.

"How to Speak It" Interview Script

"Encapsulation enforces data hiding using local and protected qualifiers. Local members are restricted to the declaring class, whereas protected members can also be accessed by derived classes. In class composition, an enclosing class instantiates and manages an internal object handle. Classes cannot contain procedural initial blocks; all instantiation and initialization must be encapsulated within the constructor."

Domain 15: Object Copying: Handle Copy vs Shallow Copy vs Deep Copy

Questions Addressed from the Repository and Practice

1. From Lesson 15 (README Questions 1, 2, 3 & 4): When is a copy needed, does new(f1) copy all data into a separate handle, and will changing p1 change f1?
2. From Lesson 16 (README Questions 1, 2, 3, 4 & 5): What is a custom copy method, and why can f1.copy be called without parentheses?
3. From Lesson 17 (README Questions 1-11): Does shallow copy duplicate nested objects, why use copy() instead of new(), and what is the relationship between constructors and copy methods?
4. From Lesson 18 (README Questions 1, 2, 3 & 5): What is wrong with shallow copy when a nested object is on the heap?
5. From Practice C9 (Lines 390-475): How do handle copy, shallow copy, and deep copy behave during state mutation?

Exhaustive Technical Explanation

Copying objects in SystemVerilog is partitioned into three distinct tiers:

1. Handle Copy (h2 = h1):

Only the handle (pointer) is copied. No new object is created on the heap. Both handles point to the exact same heap memory. Any property update via h2 is immediately visible through h1.

2. Built-in Shallow Copy (h2 = new h1):

The simulator allocates one new outer object on the heap and bitwise-copies all scalar properties from h1 into h2. **The Critical Shallow Copy Trap:** Shallow copy copies nested class handles **by reference (handle copy)**! If h1 contains a nested object handle Inner in_inst;, both h1.in_inst and h2.in_inst point to the **exact same nested object** on the heap! Mutating h2.in_inst.data corrupts h1.in_inst.data. Furthermore, shallow copy **does NOT execute the class constructor new()** or property initializers of the copied class.

3. Custom Recursive Deep Copy (h2 = h1.copy()):

To achieve complete memory independence, verification engineers implement custom copy methods. A deep copy method instantiates a new outer object and recursively calls the .copy() method of all nested objects: ``systemverilog function Outer copy(); copy = new(); copy.id = this.id; if (this.in_inst != null) copy.in_inst = this.in_inst.copy(); // Recursive clone endfunction ` This guarantees that modifying any property of h2 -- scalar or nested -- leaves h1` completely untouched.

Omitting Parentheses on Subroutine Calls: In SystemVerilog, empty parentheses can be legally omitted when calling functions or tasks with no arguments (copy2.f1 = f1.copy; is identical to f1.copy()).

"How to Speak It" Interview Script

"There are three levels of object copying in SystemVerilog. Handle copy simply creates a second pointer to the same heap object. Built-in shallow copy, using the syntax h2 = new h1, creates a new outer object and duplicates scalar properties, but it copies nested object handles by reference -- leaving nested objects shared between instances. Shallow copy also bypasses the constructor. For verification scoreboards and generators, we must implement recursive deep copy methods that explicitly instantiate and clone all nested child objects, guaranteeing total memory isolation."

Domain 16: Inheritance, Polymorphism, and Virtual Method Dispatch

Questions Addressed from the Repository and Practice

1. From Lesson 19 (README Questions 1 & 2): Does a derived class inherit attributes and methods, and can the derived handle access them directly?
2. From Lesson 20 (README Questions 1, 2 & 3): Will extending first cause the overridden display method to execute, and what does polymorphism mean?
3. From Practice C9 (Lines 560-575 & 670-685): What executes when a child object assigned to a base handle calls an overridden method?

Exhaustive Technical Explanation

Inheritance establishes an **Is-A** relationship (`class Child extends Base;`). The child class automatically inherits all non-local properties and methods of the base class.

Handle Upcasting vs Downcasting:

- **Upcasting (Implicit):** Assigning a child handle to a base class handle (`Base b = c;`) is always legal and implicit. In SystemVerilog, upcasting never slices the object; the full derived object remains intact on the heap.
- **Downcasting (Explicit via \$cast):** Assigning a base handle back into a child handle (`c = b;`) fails compilation because the compiler cannot verify whether the object on the heap is genuinely a `Child`. The verification engineer must use the `$cast(c, b)` system function, which verifies the heap object type at runtime and returns 1 on success or 0 on failure.

Polymorphism and Dynamic Virtual Method Dispatch: In Practice C9, the following comment was written:

```
f = s;          // f is Base handle, s is Child instance
f.display();    // Practice comment: "parent method will be executed"
```

CRITICAL VERIFICATION CORRECTION: THE PRACTICE COMMENT IS FALSE! Because the `display()` method was declared with the `virtual` keyword in the base class:

```
class first;
  virtual function void display();
    $display("BASE display");
  endfunction
endclass
```

SystemVerilog activates **dynamic method dispatch**. When `f.display()` is called, the simulator looks past the declared static type of the handle (`first`) and examines the **actual runtime object on the heap** (which is `second`). Therefore, `second.display()` **executes, NOT the parent method!** If `virtual` had been omitted, static compile-time binding would have executed `first.display()`. Marking methods `virtual` is the cornerstone of polymorphic testbench architectures (such as UVM).

"How to Speak It" Interview Script

"Polymorphism enables a single base-class handle to invoke specialized subclass behaviors at runtime. When a base method is declared with the virtual keyword, SystemVerilog uses dynamic method dispatch: the simulator evaluates the actual object type residing on the heap rather than the static handle type. Upcasting derived objects into base handles allows verification environments to pass transactions generically while executing overridden child behavior seamlessly."

Domain 17: Constructor Chaining and the `super` Keyword

Questions Addressed from the Repository and Practice

1. From Lesson 21 (README Questions 1, 2 & 3): Which keyword calls the parent constructor, can a constructor have output arguments, and is the constructor name always `new`?
2. From Practice C9 (Lines 580-598): How must `super.new` be placed in derived constructors?

Exhaustive Technical Explanation

When a derived class extends a base class, constructor chaining ensures that the base portion of the object is allocated and initialized before derived class properties are initialized.

- **The `super` Keyword:** The derived class constructor invokes the parent constructor using `super.new(...)`.
- **Strict Placement Rule:** The call to `super.new(...)` **MUST** be the **first executable statement** inside the derived class constructor. Placing procedural code before `super.new(...)` is illegal and causes a compile error.

- **Implicit Behavior:** If `super.new(...)` is omitted, the SystemVerilog compiler automatically injects an implicit parameterless call `super.new()` as the first line. If the base class constructor requires arguments without defaults, this implicit call fails compilation.
- **Constructor Rules:** Constructor names in SystemVerilog are strictly always `new`. Constructors can only take input arguments; declaring output or inout arguments on a constructor is illegal.

"How to Speak It" Interview Script

"In derived classes, constructor chaining guarantees hierarchical initialization. The derived constructor must invoke `super.new` as its very first executable statement, passing any required arguments to the base class. If omitted, the compiler injects an implicit zero-argument `super.new` call. Constructors can accept input arguments with default values, but output and inout ports are strictly illegal on constructors."

Domain 18: Constrained Randomization, Error Handling, and Lifecycle

Questions Addressed from the Repository and Practice

1. From Lesson 22 to 30: How do `rand` and `randc` operate, and how do constraints shape stimulus?

2. From User Practice Snippet (Generator Class):

- What is the role of a transaction class holding input and output?
- How can randomization fail in SystemVerilog?
- Why is `int status = 0;` placed after `g = new();` marked illegal in procedural blocks?
- How does range exclusion using `!(variable inside {[min:max]})` work?
- Why must `assert(g.randomize()) else $finish;` be used instead of unchecked randomization?
- What is the consequence of `g = new();` inside the loop versus reusing a single handle?

Exhaustive Technical Explanation

1. The Transaction Class (class generator / class transaction):

In modular testbenches, transaction classes encapsulate both input stimulus pins (driven to the DUT) and expected output pins (observed from the DUT). A transaction represents a discrete protocol frame or bus cycle that travels along the verification pipeline: Generator $\rightarrow$ Driver $\rightarrow$ DUT $\rightarrow$ Monitor $\rightarrow$ Scoreboard.

2. How Randomization Can Fail:

Randomization via `g.randomize()` returns 1 on success and 0 on failure. Randomization fails when: - **Contradictory Constraints:** Mutually exclusive rules (e.g., `constraint c { a > 10; a < 5; }`). - **Empty Solution Space:** Exclusions eliminating all possible values within an enumerated or bounded type (e.g., a 2-bit variable constrained `!(a inside {[0:3]})`). - **Solver Timeouts or Over-Constrained Systems:** Complex circular dependency equations exceeding solver limits. - **Null Handle Invocation:** Attempting to randomize an unconstructed handle (`g = null; g.randomize();`) causes a fatal runtime crash.

3. Declaration Order Within Procedural Blocks:

In the user's practice snippet: ```systemverilog initial begin g = new(); // Executable procedural statement int status = 0; // Declaration after a statement end`` In traditional Verilog and strict compilation modes, all variable declarations within a `begin...end` block must precede any procedural executable statements. Modern SystemVerilog relaxes this within block scopes, but variables declared after procedural statements have lifetime and visibility only from their declaration point downward. In legacy tools or strict compiler flags, mixing declarations after executable statements triggers syntax errors. The standard best practice is to declare all local variables at the beginning of the block scope before any procedural statements.

4. Range Exclusion Syntax:

The `inside` operator tests set membership: `a inside {[3:7]}` is true if `$3 <= a <= 7`. To exclude ranges, SystemVerilog applies the logical negation operator `!`: ```systemverilog constraint data { !(a inside {[3:7]}); // 'a' will never take values 3, 4, 5, 6, 7 !(b inside {[5:9]}); // 'b' will never take values 5, 6, 7, 8, 9 }` The constraint solver uniformly selects values across the remaining valid domain of the random variable.

5. Why `assert(g.randomize()) else $finish;` is Mandatory:

If `g.randomize()` fails and the testbench does not inspect the return status, the variables retain their previous stale values, and simulation continues silently. The testbench then drives corrupted, non-random stimulus into the DUT, wasting CPU hours and masking design bugs. Enclosing randomization inside an immediate assertion (`assert(g.randomize()) else $fatal(1, "Randomization failed!");`) halts simulation instantly upon solver failure.

6. Fresh Object Instantiation (`g = new();`) Inside the Stimulus Loop:

When generating stimulus inside a loop: ``systemverilog for(int i = 0; i < 10; i++) begin g = new(); // Allocates a fresh, independent heap object for each transaction assert(g.randomize()); mailbox.put(g); // Pushes independent handle to driver end `` If `g = new();` was placed ***outside*** the loop, only a single heap object would ever be allocated. Every iteration would randomize and overwrite that same object. Any downstream queue, mailbox, or scoreboard holding handles would find that all stored transactions now point to the identical last-randomized values! Instantiating `new()` on every iteration guarantees complete transaction isolation.

"How to Speak It" Interview Script

"In constrained-random verification, transaction classes bundle inputs and outputs into discrete data packets. Calling randomize() without status checking is a major verification hazard because solver failures will silently drive stale data into the DUT; we always wrap randomization in an immediate assertion: assert(trans.randomize()) else \$fatal. Range exclusions are implemented using logical negation of the inside operator. Finally, in stimulus generation loops, instantiating new() on every iteration is critical to ensure that downstream driver and scoreboard queues receive distinct, independent objects rather than multiple handles referencing a single overwritten instance."

Part I: Data Types, Representation & Simulation Foundations

Section 1: Scalar Data Types, Signedness, and Value Systems

In SystemVerilog, a **data type** defines the domain of legal values and the legal operations that can be performed upon an instance of that type. A **data object** is a concrete named entity that possesses a specific type: variables, nets, or constants.

A **variable** stores its last procedurally assigned value; it retains this value persistently across simulation time until another active assignment overwrites it. A **net** (such as `wire`) does not store state; rather, it models physical connectivity and computes its instantaneous value from the continuous resolution of all active drivers connected to it.

```
// Variables: retain procedurally assigned values across simulation time
logic [7:0] state_reg; // 4-state variable
bit [7:0] count_val; // 2-state variable

// Net: continuously resolves physical driver connectivity
wire [7:0] bus_data; // 4-state net
```

Integral Data Types: Comparative Taxonomy

The following reference table directly contrasts the SystemVerilog integral data types across simulation value states, declared bit widths, default signedness conventions, and uninitialized values:

Type Keyword	Simulation States	Bit Width	Default Signedness	Default Uninitialized Value	Primary Verification Use Case
bit	2-state (0, 1)	1 (or declared vector [m:n])	Unsigned	0 (or '0)	High-performance testbench stimulus and cycle-accurate modeling
logic / reg	4-state (0, 1, X, Z)	1 (or declared vector [m:n])	Unsigned	X (or 'x)	RTL modeling, DUT interface ports, and X-propagation checking
byte	2-state (0, 1)	8 bits	Signed	8'd0	Byte-oriented protocol packets (Ethernet, PCIe, UART, SPI)
shortint	2-state (0, 1)	16 bits	Signed	16'd0	Half-word transfers, 16-bit register modeling
int	2-state (0, 1)	32 bits	Signed	32'd0	Loop indices, transaction identifiers, general integer arithmetic

Type Keyword	Simulation States	Bit Width	Default Signedness	Default Uninitialized Value	Primary Verification Use Case
longint	2-state (0, 1)	64 bits	Signed	64'd0	64-bit addresses, high-capacity packet byte counters
integer	4-state (0, 1, X, Z)	32 bits	Signed	32'shxxxx_xxxx	Legacy Verilog loop variable modeling
time	4-state (0, 1, X, Z)	64 bits	Unsigned	64'hxxxx_xxxx_xxxx_xxxx	Simulation time storage (scaled integer format)
real / realtime	Double-precision float	64-bit IEEE 754	N/A	0.0	Analog modeling, jitter measurement, floating-point timestamps

Critical Language Traps and Repairs from Practice C1 & C2

1. The bit Default Signedness Trap (Practice C1):
In user practice C1 line 4, the comment stated: `// bit is a 2 state variable type , signed.` **Correction:** bit is **unsigned by default**. A single bit `a`; can hold only 0 or 1. To make a bit vector signed, the keyword `signed` must be explicitly supplied: `bit signed [7:0] s_bit;`.
2. Uninitialized 4-State Defaults (Practice C4):
In user practice C4 line 56, the comment stated: `// 4 state -> initialized to unknown (Z).` **Correction:** Uninitialized 4-state variables (`logic`, `reg`, `integer`) initialize strictly to **X (unknown)**, NEVER **Z** (high impedance)! High impedance **Z** models an undriven physical net; it only appears if explicitly assigned (`bus = 1'bz;`) or when a tri-state driver is turned off.
3. Signedness vs Bit Width: Decimal Interpretation (Practice C2):
Practice C2 demonstrates `byte var_val = -126;` and `bit [7:0] var2 = 130;`. Both variables store the identical 8-bit binary pattern: `8'b1000_0010` (8'h82). - For `byte`, the MSB (bit 7) is interpreted as a two's-complement sign bit with weight -128. Thus: $-128 + 2 = -126$. - For `bit [7:0]`, the MSB has weight +128. Thus: $+128 + 2 = 130$. - When assigning a value wider than 8 bits (e.g. `300 = 9'b1_0010_1100`), the excess most-significant bits are truncated, preserving `8'h2c` (decimal 44).
4. Reserved Identifier Trap: `var` (Practice C2):
In user practice C2, the identifier `var` is used as a variable name: `byte var = -126;`. **Correction:** `var` is a reserved SystemVerilog declaration keyword (e.g. `var logic [7:0] bus;`). It cannot be used as an identifier. Standard tools reject this with a syntax error.

Section 2: Simulation Time, Timescale, Scheduling, and Clock Generation

The ``timescale` Directive: Unit vs Precision

- Simulation time is governed by the compiler directive ``timescale <time_unit> / <time_precision> ```:
- `time_unit`: Determines the physical time represented by an unsuffixed delay. For example, under ``timescale 1ns/1ps ```, `#10` represents 10 nanoseconds.
 - `time_precision`: Determines the scheduling quantum and rounding boundary within the simulation engine. All event times are rounded to the nearest multiple of the precision before scheduling. Precision is not merely a display formatting rule; it dictates physical simulation granularity.

Time Query Functions: \$time, \$realtime, and \$timeformat

```
module time_sampling_demo;
    timeunit 1ns;
    timeprecision 1ps;

    time    t_int;
    realtime t_real;

    initial begin
        $timeformat(-9, 3, " ns", 10);
        #12.234;
        t_int = $time;    // returns 64-bit rounded integer: 12
        t_real = $realtime; // returns real-valued timestamp: 12.234

        $display("Integer time: %0d (formatted: %0t)", t_int, t_int);
        $display("Realtime time: %0f (formatted: %0t)", t_real, t_real);
    end
endmodule
```

Key Distinction: Variables declared as time or realtime are static storage objects; they do **not** automatically track simulation time. They update only upon explicit assignment (`t_real = $realtime;`).

Testbench Clock Generation and Period Verification (Parts 02 & 03)

In testbench verification, clock generation processes must be derived from precise period equations:

$$T_{\text{Period (ns)}} = \frac{1000}{F_{\text{Frequency (MHz)}}}$$

```
// 100 MHz clock (10 ns period, 50% duty cycle)
always #5 clk_100 = ~clk_100;

// Correct 25 MHz clock (40 ns period, 20 ns high / 20 ns low)
always begin
    #20 clk_25 = 1'b1;
    #20 clk_25 = 1'b0;
end
```

Bug Audit from Part 02: In GitHub Part 02, the signal named `clk25Mhz` was defined with:

```
always begin
    #20 clk25Mhz = 1'b1;
    #10 clk25Mhz = 1'b0;
end
```

Analysis: The total period is $20\text{ ns} + 10\text{ ns} = 30\text{ ns}$. The resulting frequency is $1000 / 30 = 33.33\text{ MHz}$, NOT 25 MHz! To obtain a true 25 MHz clock, both high and low phases must be 20 ns ($T = 40\text{ ns}$).

Phase Shift vs Time-Zero Startup Offset (Part 03): An initial delay `#10` prior to starting a clock loop introduces a startup offset from simulation time zero. A true phase shift describes the steady-state like-edge delay relative to a reference clock grid.

Section 3: Arrays: Packed, Unpacked, and Memory Layouts

```
Packed Array:    logic [3:0][7:0] packed_reg; (One contiguous 32-bit register word)
                  [ Byte 3 ][ Byte 2 ][ Byte 1 ][ Byte 0 ]

Unpacked Array:  logic [7:0] unpacked_mem [4]; (Four distinct 8-bit memory locations)
                  [ Element 0: 8 bits ]
                  [ Element 1: 8 bits ]
                  [ Element 2: 8 bits ]
                  [ Element 3: 8 bits ]
```

- **Packed Array:** Dimension is placed **before** the variable name (bit `[7:0] bus;`). It is guaranteed to be stored as a single contiguous vector of bits in simulator memory. It supports vector operations, arithmetic, and bit-slicing.
- **Unpacked Array:** Dimension is placed **after** the variable name (bit `bus [8];` or `int arr [0:9];`). It represents an indexed collection of independent elements.

- **Dynamic Array:** Unpacked array declared with empty brackets (`int dyn[];`). At declaration, size is 0 and no heap memory is allocated. Memory must be allocated dynamically using the `new[N]` constructor.

Array Initialization Constructs (Practice C3, C4 & Part 05)

SystemVerilog provides two distinct aggregate initialization constructs:

1. Unpacked Array Concatenation (`{...}`):

Legal when assigned to an unpacked array target. For example, `bit arr[] = {1, 0, 1, 1};` assigns four elements directly without requiring an apostrophe.

2. Assignment Patterns (`'{...}`):

The primary, robust SystemVerilog method for structured data initialization: ```systemverilog int a[5] = '{1, 3, 4, 2, 2};` // Positional assignment
`int b[5] = '{5{0}};` // Pattern replication (5 copies of 0)
`int c[5] = '{default: 4};` // Default value for all unassigned elements
`int d[9] = '{1, 2, default: 0};` // Positional prefix with default tail ``

Array Size Querying Functions

- `$size(arr [, dim])`: Returns the number of elements in the specified dimension of fixed or dynamic arrays.
- `$bits(expression)`: Returns the total number of bits required to represent the bit-stream of the object.
- `arr.size()`: Built-in method returning the current element count of dynamic arrays and queues.
- **Rejection of `$sizeof()`:** In practice C3 line 46, `$sizeof(arr)` is written. `$sizeof` is a C/C++ keyword; it does not exist in SystemVerilog. Attempting to use `$sizeof` causes a syntax error.

Section 4: Array Iteration and Loop Mechanics

SystemVerilog features five primary loop constructs for procedural execution, each tailored to specific verification requirements:

1. The `for` Loop

The standard three-clause loop (`for(initialization; condition; modifier)`) allows explicit index control. Modern SystemVerilog permits declaring the loop index variable locally within the initialization clause: `for(int i = 0; i < 10; i++)`. The variable `i` is local to the loop and destroyed upon exit.

2. The `foreach` Loop

The `foreach(arr[idx])` loop is designed specifically for array traversal. The loop index `idx` is automatically declared with local scope and is strictly read-only. The simulator automatically infers the low and high bounds from the array declaration, safely iterating across ascending, descending, or non-zero low bounds without indexing defects.

3. The `repeat` Loop

The `repeat(count)` construct executes its procedural block a fixed number of times. The loop count expression is evaluated once upon entry. `repeat` is the industry standard loop inside transaction generator classes for generating a deterministic count of random stimuli.

4. The `while` and `do...while` Loops

The `while(condition)` loop tests its Boolean condition prior to executing the loop body, making it ideal for polling signals where zero iterations may be required. The `do ... while(condition)` loop executes the body at least once before testing the condition, which is ideal for handshake protocols where an initial request must be asserted before checking for an acknowledgment.

5. The `forever` Loop

The `forever begin ... end` construct executes an infinite procedural loop. It must always contain an internal timing control (`#delay, @(posedge clk), wait`) to prevent simulation thread starvation.

The Golden Law of Simulation Time in Loops

CRITICAL LAW: *Iteration alone consumes ZERO simulation time!*

Executing a loop of 1,000 iterations takes 0 nanoseconds on the simulation timeline unless an explicit timing control (`#delay, @(posedge clk), wait`) is reached inside the loop body.

In user practice C5, running three concurrent `initial` blocks writing to the same array without synchronization introduces an unscheduled race condition at simulation time zero.

Section 5: Array Operations: Assignment, Comparison, and Dynamic Resizing

Whole-Array Assignment and Equality (Practice C6 & Part 07)

SystemVerilog supports whole-array assignments:

```
int arr1[5];
int arr2[5];
// ... initialize arr1 ...
arr2 = arr1; // Copies all 5 element values independently
```

- **Assignment Semantics:** Array assignment is a **value copy**. Modifying `arr2[0]` after assignment does not alter `arr1[0]`.
- **Equality Comparison (`==` vs `===`):**
- `arr1 == arr2`: Compares element values logically. If any element contains `x` or `z`, the result is `1'bx`.
- `arr1 === arr2`: Case equality. Treats `x` and `z` as literal bit states and returns a deterministic Boolean `1'b1` or `1'b0`.

Dynamic Array Memory Management (Practice C7)

Dynamic arrays are allocated and resized via the dynamic-array new-constructor `new[N]`:

```
int dyn[];
dyn = new[5];           // Allocates 5 elements: '{0, 0, 0, 0, 0}'
foreach(dyn[i]) dyn[i] = i*i; // Initializes to: '{0, 1, 4, 9, 16}'

// Resizing with data preservation:
dyn = new[8](dyn);       // Preserves original 5 elements, pads 3 zeros:
                        // '{0, 1, 4, 9, 16, 0, 0, 0}'

// Deallocation:
dyn.delete();           // Frees heap memory; size becomes 0
```

- If `newZZTOKEN0ZZ` shrinks the array, only the first `$N$` elements are preserved.
- When copying a dynamic array into a fixed-size array (`fixed_arr = dyn;`), the dynamic array's current size must **exactly match** the fixed array's dimension, otherwise a fatal runtime error occurs.

Section 6: Queues: Syntax, Operations, and Performance

A **queue** is an unbounded or bounded ordered collection with index range `0` to `$`. Declared with `[ $ ]`:

```
int q[$];               // Unbounded queue
int bounded_q[$:127];   // Bounded queue of maximum 128 elements
```

Why Queues Excel in Verification Testbenches

Unlike dynamic arrays, which require full re-allocation and memory copying when resized, queues are implemented using contiguous segmented memory blocks. Queues provide $O(1)$ constant-time insertion and deletion at both front and back boundaries. This makes queues ideal for FIFO modeling, driver-to-monitor pipelines, and transaction scoreboards.

Comprehensive Architecture of Built-in Queue Methods

SystemVerilog equips queues with built-in manipulation methods that manage memory dynamically:

- `size()`: Returns the current element count as an integer: `int count = q.size();`
- `push_front(val)`: Inserts `val` at index `0`, shifting existing elements up by one index in $O(1)$ time.
- `push_back(val)`: Appends `val` at index `$` (end of queue) in $O(1)$ time.
- `pop_front()`: Removes and returns the element at index `0`. If the queue is empty, a warning is issued and the type default value is returned.
- `pop_back()`: Removes and returns the element at index `$`.
- `insert(idx, val)`: Inserts `val` immediately before index `idx`. Requires two arguments: index and value.
- `delete(idx)`: Removes the element at index `idx` and shifts subsequent elements left.
- `delete()`: Clears all elements from the queue, resetting its size to `0`.

Execution Trace of Practice C8 Operations

```
int q[$];
int j, k;

q = {1, 3, 4};           // Initial state:           '{1, 3, 4}'
q.push_front(7);        // Insert 7 at index 0:       '{7, 1, 3, 4}'
q.push_back(9);         // Append 9 at end:         '{7, 1, 3, 4, 9}'
q.insert(2, 8);          // Insert 8 at index 2:     '{7, 1, 8, 3, 4, 9}'
j = q.pop_front();       // Pops 7; queue becomes:   '{1, 8, 3, 4, 9}'
k = q.pop_back();        // Pops 9; queue becomes:   '{1, 8, 3, 4}'
q.delete(1);            // Deletes element at idx 1 (8): '{1, 3, 4}'
```

(Note: Practice C8 wrote `arr.insert(2)` missing the second argument. `insert()` requires both index and value).

Part II: Subroutines & Argument Passing Semantics

Section 7: Tasks vs Functions

Subroutines package procedural algorithms into reusable units. The selection between a task and a function is governed strictly by the following architectural language boundaries:

1. Simulation Time Consumption

A **function** is strictly forbidden from consuming simulation time. It cannot contain blocking delay statements (`#10`), event timing controls (`@(posedge clk)`), wait statements, or blocking task calls. It executes instantaneously within a single evaluation step. A **task** is capable of suspending execution, waiting for clock edges, and consuming simulation time.

2. Return Values and Invocation Contexts

A non-void function returns a value directly, allowing it to be evaluated inside expressions, conditional tests (`if (calc_parity(data))`), and continuous assignments. A task does not return a value via expression syntax; it is called strictly as a standalone procedural statement (`send_stimulus(pkt);`).

3. Calling Hierarchies

A task can call both other tasks and functions freely. A function can call other functions, but a function **cannot call a task** because doing so would jeopardize the function's zero-simulation-time guarantee (unless the task is spawned in an independent background thread via `fork...join_none`).

Refuting the Common Myth: "Functions Cannot Have Output Ports"

In user practice C9 lines 168--169, the comment asserts: `// function , not supports output ports`. **Correction:** According to IEEE Std 1800-2017 Clause 13.4, **procedural functions in SystemVerilog fully support** output, inout, and ref arguments! Functions with output arguments cannot be called inside non-procedural contexts (such as continuous assignments or event expressions), but inside procedural blocks (`initial`, `always`, `tasks`), a function with output arguments is 100% legal:

```
function void compute_stats(input int data, output int square, output int cube);
    square = data * data;
    cube   = data * data * data;
endfunction
```

Section 8: Argument Passing Modes & "How to Speak It" Interview Scripts

SystemVerilog provides five argument passing qualifiers with distinct memory semantics:

1. input Mode (Pass-by-Value)

Upon subroutine entry, the simulator allocates a dedicated storage slot on the local stack frame and copies the caller's value into it. Any modifications made to the formal argument alter only this local copy. The caller's variable remains completely unchanged upon return.

2. output Mode (Pass-by-Value Return)

The formal argument begins initialized to the default value of its type. Inside the subroutine, code assigns values to the formal argument. Upon normal return from the subroutine, the final value of the formal argument is copied back into the caller's actual variable.

3. inout Mode (Bidirectional Pass-by-Value)

Combines input and output: the caller's value is copied into the local formal argument upon entry, and the final modified local value is copied back out to the caller's variable upon return.

4. ref Mode (Pass-by-Reference)

The simulator passes a direct memory alias to the caller's actual variable. No local copy is created on the stack. Any assignment made to a ref formal argument is instantly reflected in the caller's variable in real time. Passing large unpacked arrays or structures by ref eliminates stack-copying performance penalties.

5. const ref Mode (Read-Only Reference)

Combines the performance advantage of memory aliasing with strict read-only safety. The formal argument aliases the caller's memory directly, but the compiler forbids any modification to the formal argument within the subroutine body.

Verbatim Interview Script: How to Explain Pass-by-Value

"How to Speak Pass-by-Value" (Verbatim Interview Guide):

"In SystemVerilog, pass-by-value is the default mechanism for input arguments. When an argument is passed by value, the simulator allocates a private storage location for the formal argument on the subroutine's execution stack and copies the actual argument's value into it upon entry.

Any subsequent modifications made to this formal argument inside the subroutine operate purely on this local copy. The caller's original variable remains completely isolated and unchanged throughout the call.

Pass-by-value guarantees total data protection for the caller, but it incurs a performance and memory penalty when large data structures -- such as unpacked arrays -- are repeatedly copied across the stack."

Verbatim Interview Script: How to Explain Pass-by-Reference

"How to Speak Pass-by-Reference" (Verbatim Interview Guide):

"Pass-by-reference, specified using the ref keyword, passes a direct reference -- or memory alias -- to the caller's actual variable rather than copying its contents.

Because no copy is created, any assignment made to a ref formal argument is immediately reflected in the caller's variable in real time, without waiting for the subroutine to finish and return.

In verification environments, pass-by-reference is essential for two primary reasons: first, to achieve high-performance stimulus generation by avoiding expensive stack copying of large arrays and packets; and second, to allow concurrent tasks to inspect or modify shared state dynamically.

Crucially, SystemVerilog requires that subroutines with ref arguments in static scopes be declared with the automatic lifetime keyword to guarantee thread-safe call frames."

The automatic Keyword and Storage Lifetimes

- **Static Lifetime:** Variables allocated once at time zero and shared across all invocations. Module- and interface-level subroutines default to static.
- **Automatic Lifetime:** Variables allocated dynamically on the call stack for each invocation. Class methods default to automatic.
- **Strict Rule (IEEE 13.5.2):** A subroutine that declares a ref argument **MUST be declared** automatic if defined inside a module or interface!

Passing Class Handles: Value of Handle vs Mutating the Object

A frequent interview trap involves passing a class handle as an input argument:

```
function automatic void modify_pkt(input Packet p);
    p.id = 99;      // MUTATES the caller's shared object on the heap!
    p  = null;     // NULLS ONLY the local copy of the handle!
endfunction
```

Explanation: Passing a class handle as input copies the **pointer (handle)** by value. Both the caller's handle and the callee's local handle point to the exact same object in heap memory. Modifying an internal property (`p.id = 99;`) modifies the shared object. However, reassigning the handle itself (`p = null;`) merely modifies the callee's local stack copy; upon return, the caller's handle remains non-null and points to the mutated object!

Part III: Object-Oriented Programming (OOP) in SystemVerilog

Section 9: Classes, Handles, Objects, and Lifecycle

SystemVerilog OOP separates **type**, **handle**, and **heap object**:

```
Class Declaration (Blueprint): class Transaction; int addr; endclass

Handle Declaration:           Transaction tr;           tr ----> [ null ]
(Stack Pointer)

Object Construction:          tr = new();               tr ----> [ Heap Object: addr = 0 ]
```

1. **Class Type:** A user-defined data type defining properties (data) and methods (tasks/functions).
2. **Handle Variable:** A typed pointer variable that holds either null or a reference to an active object on the heap. Default value is strictly null.
3. **Object:** Dynamic memory allocated on the simulation heap via new().
4. **Dereferencing null:** Attempting to access members of an unconstructed handle (tr.addr = 5; when tr == null) causes an immediate fatal simulation runtime crash.
5. **Handle Assignment:** tr2 = tr1; copies the **handle**, not the object! Both handles now reference the exact same memory block.
6. **Automatic Garbage Collection:** SystemVerilog features automated heap garbage collection. Writing tr = null; merely removes that reference. The object remains alive as long as at least one handle references it. When zero handles reference the object, the simulator reclaims the heap memory.

Section 10: Constructors, this, Encapsulation, and Composition

The Class Constructor (new)

The constructor is declared with function new(...).

- It has **no return type** (not even void).
- It allocates memory, initializes properties to default values, and returns the handle implicitly.
- It supports default argument values, enabling flexible invocation with positional or named arguments:

```
``systemverilog class Packet; int length; int payload; function new(int length = 64, int payload = 0); this.length = length; // 'this'
disambiguates property from argument this.payload = payload; endfunction endclass
```

```
Packet p1 = new(); // Uses defaults: length=64, payload=0
Packet p2 = new(128, 5); // Positional arguments
Packet p3 = new(.payload(9), .length(256)); // Named arguments ``
```

Encapsulation and Access Modifiers

- **public (default):** Properties and methods can be accessed from any scope.
- **local:** Accessible strictly from within the declaring class. Subclasses and external scopes cannot access local members.
- **protected:** Accessible from within the declaring class AND any derived subclasses (extends). External scopes cannot access protected members.

Composition (Has-A) vs Inheritance (Is-A)

- **Composition:** An outer class contains a handle to an inner class (Envelope has a Payload). The outer constructor is responsible for constructing the nested object (payload = new();).
- **Language Law:** A class **cannot contain an initial block!** Initialization belongs inside the new() constructor.

Section 11: Object Copying Mechanisms: Handle Copy vs Shallow Copy vs Deep Copy

```

1. Handle Copy:    h1 --+
                   +----> [ Object Instance A ]
                   h2 --+

2. Shallow Copy:   h1 -----> [ Outer Object A ] -----+
                   +-----> [ Nested Object N ]
                   h2 -----> [ Outer Object B ] -----+

3. Deep Copy:       h1 -----> [ Outer Object A ] -----> [ Nested Object N1 ]
                   h2 -----> [ Outer Object B ] -----> [ Nested Object N2 ]

```

Detailed Breakdown of the Three Copying Mechanisms

1. Handle Copy ($h2 = h1$;) In a handle copy, zero new objects are instantiated on the heap. The assignment simply copies the pointer address from $h1$ into $h2$. Both handles now alias the exact same physical memory block. Any mutation performed via $h2$ immediately updates the state observed by $h1$.

2. Built-in Shallow Copy ($h2 = \text{new } h1$;) In a shallow copy, the simulator creates exactly one new object on the heap for $h2$. All scalar properties (`int`, `bit`, `logic`) are copied by value, giving $h2$ independent scalar state. **The Shallow Copy Hazard:** If the class contains handles to nested objects, those nested handles are copied **by reference**! Therefore, both $h1$ and $h2$ point to the identical nested object on the heap. Any mutation of the nested object via $h2$ corrupts $h1$! Furthermore, shallow copy **does NOT execute the class constructor** `new()` or property initializers of the copied class.

3. Custom Recursive Deep Copy ($h2 = h1.\text{copy}()$;) In a custom deep copy, the verification engineer implements a copy method that instantiates a new outer object, copies all scalar fields, and explicitly invokes `.copy()` on every nested child object. This produces complete hierarchical memory independence.

Implementation of Recursive Deep Copy

```

class Inner;
  int data = 10;
  function Inner copy();
    copy = new();
    copy.data = this.data;
  endfunction
endclass

class Outer;
  int id = 1;
  Inner nested;

  function new();
    nested = new();
  endfunction

  function Outer copy();
    copy = new();
    copy.id = this.id;
    if (this.nested != null)
      copy.nested = this.nested.copy(); // Recursive clone of nested child
    else
      copy.nested = null;
  endfunction
endclass

```

Step-by-Step State Mutation Trace Walkthrough

Consider an initial state where $s1.id = 1$ and $s1.nested.data = 10$:

1. Under Handle Copy ($s2 = s1$):

We execute $s2.id = 2$; $s2.nested.data = 99$;. Because $s1$ and $s2$ are two handles pointing to the exact same heap object, inspecting $s1$ reveals $s1.id = 2$ and $s1.nested.data = 99$. Both handles see all mutations.

2. Under Built-in Shallow Copy ($s2 = \text{new } s1$):

We execute $s2.id = 2$; $s2.nested.data = 99$;. Because `id` is a scalar, $s1.id$ remains 1 while $s2.id$ becomes 2. However, because `nested` is a shared handle, $s1.nested.data$ is corrupted to 99!

3. Under Custom Deep Copy (`s2 = s1.copy()`):

We execute `s2.id = 2; s2.nested.data = 99;`. Because `s1.nested` and `s2.nested` reference distinct heap objects, `s1.id` remains 1 and `s1.nested.data` remains 10. Total isolation is achieved.

Section 12: Inheritance, Polymorphism, Virtual Methods, and Constructor Chaining

Inheritance Syntax and Handle Upcasting

Inheritance models an **Is-A** relationship (`class Child extends Base;`). Derived classes inherit all properties and methods of the base class.

- **Upcasting (Derived to Base):** Assigning a child handle to a base handle (`Base b = child_inst;`) is always legal and implicit. In SystemVerilog, upcasting never slices the object; the underlying heap object remains a complete `Child`.
- **Downcasting (Base to Derived):** Assigning a base handle to a child handle (`Child c = b;`) fails compilation. Safe runtime downcasting requires `$cast(c, b)`:

```
``systemverilog if (!$cast(c, b)) $fatal(1, "Downcast failed: underlying object is not of type Child!"); ``
```

Polymorphism and Virtual Method Dispatch

```
class Base;
    virtual function void display();
        $display("Executing BASE display");
    endfunction
endclass

class Child extends Base;
    function void display();
        $display("Executing CHILD display");
    endfunction
endclass

module tb;
    Base b;
    Child c;

    initial begin
        c = new();
        b = c;        // Upcast child handle into base handle
        b.display();  // Which display() executes?
    end
endmodule
```

Resolution of the Practice Code Misconception (L569 & L676)

CRITICAL CODE AUDIT & CORRECTION:

In the user's practice code, the following comment was written:

```
``systemverilog
f = s; // what this do besides copying the handler of class s to f
f.display(); // parent method will be executed
...

```

VERIFICATION VERDICT: THE COMMENT IS INCORRECT!

*Because `display()` was declared with the `virtual` keyword in the base class, SystemVerilog uses **dynamic method dispatch**.*

*The simulator looks past the static type of the handle (`f` is declared `Base`) and checks the **dynamic type of the object in heap memory** (which is `Child`).*

*Therefore, `Child.display()` **executes, NOT the parent method!***

If the base method lacked `virtual`, static binding would execute `Base.display()`. Marking the method `virtual` guarantees that the child override executes.

Constructor Chaining and the `super` Keyword (Part 21)

When a derived class inherits from a base class with a parameterized constructor, the derived class constructor must explicitly call `super.new(...)`:

```

class Base;
  int base_id;
  function new(int id);
    this.base_id = id;
  endfunction
endclass

class Child extends Base;
  int child_val;
  function new(int id, int val);
    super.new(id);          // MUST be the first executable statement!
    this.child_val = val;
  endfunction
endclass

```

- **Strict Placement Rule:** `super.new(...)` must be the **first executable statement** inside the derived constructor.
- **Implicit Behavior:** If `super.new(...)` is omitted, the compiler automatically inserts an implicit zero-argument call `super.new()`. If the base class constructor requires arguments without defaults, compilation fails immediately.

Section 13: Constrained Randomization Fundamentals, Error Handling & Lifecycle

Constrained randomization is the core engine of functional verification. Stimulus generation models real-world input transactions while targeting corner-case scenarios.

The Transaction Class Architecture

In verification architectures, transaction classes package stimulus inputs (e.g. addresses, payloads) and expected DUT outputs into a single coherent object. Transactions are generated by the Generator, serialized by the Driver, sampled by the Monitor, and validated by the Scoreboard.

```

class generator;
  rand bit [3:0] a, b;
  bit [3:0] y;

  // Inverted range exclusion constraints
  constraint data {
    !(a inside {[3:7]}); // Exclude values 3 through 7
    !(b inside {[5:9]}); // Exclude values 5 through 9
  }
endclass

```

How Randomization Can Fail

Calling `g.randomize()` returns 1 on success and 0 on failure. Randomization fails when:

1. **Contradictory Constraints:** Mutually exclusive assertions (e.g. $a > 10$ and $a < 5$).
2. **Empty Domain Space:** Range exclusions that eliminate all legal values of a bounded variable (e.g., a 2-bit variable with `!(a inside {[0:3]})`).
3. **Solver Timeouts:** Mathematical constraints with circular dependencies that exceed solver time limits.
4. **Null Pointer Invocations:** Attempting to randomize an unconstructed handle (`generator g; g.randomize();`) results in a fatal runtime crash.

Procedural Declaration Order Rules

In the user's practice code, the following block was audited:

```

initial begin
  g = new();          // Executable procedural statement
  int status = 0;     // Declaration after a statement
end

```

In standard Verilog and strict compilation modes, all variable declarations within a `begin...end` block must precede any executable procedural statements. Placing `int status = 0;` after `g = new();` triggers a syntax error. Standard verification hygiene mandates declaring all process variables at the head of the block:

```
initial begin
  int status;
  g = new();
  status = g.randomize();
end
```

Verification Rigor: Why `assert(g.randomize())` is Mandatory

Never call `g.randomize()` as an unchecked procedural statement. If randomization fails, variables retain their previous stale values, and the testbench drives corrupt data without warning. Best practice mandates enclosing randomization in an immediate assertion:

```
assert(g.randomize()) else begin
  $fatal(1, "Randomization failed at time %0t!", $time);
end
```

Object Allocation Inside Stimulus Loops: `new()` Lifecycle

In generator loops, instantiating `g = new();` inside the loop allocates a fresh heap object for every transaction:

```
initial begin
  for(int i = 0; i < 10; i++) begin
    g = new(); // Allocate fresh object on every iteration!
    assert(g.randomize()) else $fatal(1, "Solver error!");
    $display("a=%0d, b=%0d at time %0t", g.a, g.b, $time);
    #10;
  end
end
```

If `g = new();` were called only once before the loop, every iteration would re-randomize that single object. Downstream queues or mailboxes storing handles would find all stored transactions overwritten with the final loop iteration's values.

Part IV: Practice Code Bug Audit and Code Repairs

This section audits every bug, typo, syntax violation, and conceptual defect identified in the user's raw practice source (C1 through C9, OOP blocks, and generator code), providing root-cause analysis and runnable, standard-compliant SystemVerilog implementations.

Bug Dossier 1: `bit` Default Signedness Trap (Practice C1)

- **Practice Code Reference:** Line 4: `bit a = 0; // bit is a 2 state variable type , signed`
- **Defect Analysis:** The user comment assumes that `bit` is signed. In IEEE Std 1800-2017 Section 6.8, `bit` is explicitly defined as **unsigned**.
- **Corrected Implementation:**

```
``systemverilog bit a = 0; // 2-state, unsigned (0 or 1, defaults to 0) bit signed [7:0] s_byte = -12; // Explicit
'signed' required for signed bit vectors ``
```

Bug Dossier 2: Reserved Keyword Used as Identifier (Practice C2)

- **Practice Code Reference:** Line 27: `byte var = -126;`
- **Defect Analysis:** `var` is a reserved SystemVerilog keyword used in explicit variable declarations (`var logic [7:0] data;`). Using `var` as an identifier causes an immediate syntax error.
- **Corrected Implementation:**

```
``systemverilog byte var_val = -126; // Legal identifier ``
```

Bug Dossier 3: Variable Name Mismatch Typo (Practice C2)

- **Practice Code Reference:** Line 35: Variable declared as `fix_time`, but `$display` references `fixed_time`.
- **Defect Analysis:** Simulators reject undeclared identifiers during elaboration.
- **Corrected Implementation:**

```
``systemverilog time fix_time = 0; // ... $display("Fixed time: %0t", fix_time); ``
```

Bug Dossier 4: Rejection of C-Style `$sizeof()` Operator (Practice C3)

- **Practice Code Reference:** Line 46: `$display(..., $sizeof(arr));`
- **Defect Analysis:** `$sizeof()` is a C/C++ operator that does NOT exist in SystemVerilog.
- **Corrected Implementation:**

```
``systemverilog $display("Size of arr: %0d elements, %0d bits", $size(arr), $bits(arr)); ``
```

Bug Dossier 5: 4-State Default Value Misconception (Practice C4)

- **Practice Code Reference:** Line 56: `// 4 state -> initialized to unknown (Z)`
- **Defect Analysis:** 4-state integral types (logic, reg, integer) initialize strictly to X (**unknown**), NEVER Z (high impedance). Z only appears on un-driven nets or tri-state assignments.
- **Corrected Implementation:**

```
``systemverilog logic [7:0] uninit_bus; // Automatically initializes to 8'hxx ``
```

Bug Dossier 6: Undeclared Variable in Loop Scope (Practice C5)

- **Practice Code Reference:** Line 93: `repeat(10) begin arr[i] = i; i++; end`
- **Defect Analysis:** The variable `i` was not declared within the procedural block, and `$display` contained an empty argument.
- **Corrected Implementation:**

```
``systemverilog initial begin int idx = 0; repeat(10) begin arr[idx] = idx; idx++; end $display("Array values: %0p", arr); end ``
```

Bug Dossier 7: Misconception of Dynamic Array `new[N]` (Practice C7)

- **Practice Code Reference:** Line 125: `arr = new[5];` // its not a constructor , constructor is `()`
- **Defect Analysis:** IEEE Std 1800-2017 Clause 7.5 explicitly names `new[N]` the **dynamic array new-constructor**. While distinct from class constructors, it is indeed an allocation constructor.
- **Corrected Implementation:**

```
``systemverilog arr = new[5]; // Dynamic array allocation constructor arr = newZZTOKEN0ZZ; // Dynamic array resize constructor with data preservation ``
```

Bug Dossier 8: Missing Argument in Queue `insert()` Method (Practice C8)

- **Practice Code Reference:** Line 153: `arr.insert(2);`
- **Defect Analysis:** SystemVerilog's built-in `insert()` method requires two arguments: `queue.insert(index, value)`. Calling it with one argument triggers a compilation error.
- **Corrected Implementation:**

```
``systemverilog arr.insert(2, 8); // Inserts value 8 at index 2 ``
```

Bug Dossier 9: Myth that Functions Cannot Have Output Ports (Practice C9)

- **Practice Code Reference:** Line 168: `// function , not supports output ports`
- **Defect Analysis:** Procedural functions in SystemVerilog fully support output, inout, and ref arguments per IEEE Std 1800-2017 Clause 13.4.
- **Corrected Implementation:**

```
``systemverilog function void add_and_sub(input int a, b, output int sum, diff); sum = a + b; diff = a - b; endfunction ``
```

Bug Dossier 10: Missing `automatic` Lifetime for `ref` Subroutine (Practice C9)

- **Practice Code Reference:** Line 277: `task swap(ref bit [1:0] a, b);` inside a module without `automatic`.
- **Defect Analysis:** IEEE Std 1800-2017 Clause 13.5.2 mandates that any subroutine declaring `ref` arguments inside a static scope (module or interface) MUST be declared `automatic`.
- **Corrected Implementation:**

```
``systemverilog task automatic swap(ref bit [1:0] a, b); bit [1:0] temp = a; a = b; b = temp; endtask ``
```

Bug Dossier 11: Dynamic Polymorphism Misconception (Practice C9)

- **Practice Code Reference:** Line 569: `f = s; f.display();` // parent method will be executed
- **Defect Analysis:** Because `display()` was declared virtual in base class first, dynamic dispatch executes `second.display()` (the child method), NOT the parent method.
- **Corrected Implementation:**

```
``systemverilog first f; second s = new(); f = s; // Upcast f.display(); // Correctly executes CHILD method
'second.display()' due to virtual dispatch! ``
```

Bug Dossier 12: Declaration After Statement & Unchecked Randomization (Practice Generator)

- **Practice Code Reference:** Procedural block placing `int status = 0;` after `g = new();`, and unchecked `randomize()`.
- **Defect Analysis:** Declarations after executable statements violate standard block rules, and unchecked randomization permits silent simulation corruption.
- **Corrected Implementation:**

```
``systemverilog initial begin generator g; for(int i = 0; i < 10; i++) begin g = new(); assert(g.randomize()) else
begin $fatal(1, "Randomization failed at simulation time %0t!", $time); end $display("Transaction: a=%0d, b=%0d at
time %0t", g.a, g.b, $time); #10; end end ``
```

Part V: Active Recall Self-Test & Detailed Solutions

Test your recall aloud before reading the solution keys:

1. **Question 1 (Data Types):** An 8-bit variable holds binary `8'b1111_1100`. What is printed by `$display("%0d", var)` if declared as byte, and if declared as bit `[7:0]`?

Answer: byte prints -4 (signed two's-complement: $-\$128 + 124 = -4\$$). bit `[7:0]` prints 252 (unsigned magnitude: $\$240 + 12 = 252\$$).

2. **Question 2 (Initial Values):** What are the default values of an uninitialized bit, logic, int, time, and class handle?

Answer: bit: 0. logic: X. int: 0. time: X. Class handle: null.

3. **Question 3 (Time Precision):** Under ``timescale 1ns/10ps``, what does `#1.237` delay for? What do `$time` and `$realtime` return?

Answer: Precision is 10 ps (0.01 ns). `#1.237` is rounded to 1.24 ns. `$time` returns scaled integer 1; `$realtime` returns real 1.240.

4. **Question 4 (Array Queries):** For logic `[7:0] mem [16];`, what do `$size(mem)` and `$bits(mem)` return?

Answer: `$size(mem)` returns 16 (number of unpacked elements). `$bits(mem)` returns 128 ($\$16 \times 8\$$ total bits).

5. **Question 5 (Loops and Time):** Does a for loop of 10,000 iterations advance simulation time?

Answer: No. Repetition consumes zero simulation time. Time advances only upon encountering blocking timing controls (`#`, `@`, `wait`).

6. **Question 6 (Dynamic Arrays):** A dynamic array `dyn` holds `{10, 20, 30}`. What does `dyn = newZZTOKEN0ZZ;` do? What does `dyn.delete();` leave?

Answer: `newZZTOKEN0ZZ` preserves the 3 elements and pads 2 default zeros: `{10, 20, 30, 0, 0}`. `dyn.delete()` frees memory and resets size to 0 (empty).

7. **Question 7 (Queue Methods):** Starting from `q = {10, 30}`, apply `q.push_front(5)`, `q.push_back(40)`, `q.insert(2, 20)`, `q.pop_front()`. What remains in `q`?

Answer: Sequence: `{5, 10, 30}` $\rightarrow$ `{5, 10, 30, 40}` $\rightarrow$ `{5, 10, 20, 30, 40}` $\rightarrow$ pop 5 leaves `{10, 20, 30, 40}`.

8. **Question 8 (Functions and Tasks):** Why can a function not call a task directly? Can a function have an output argument?

Answer: A function cannot consume simulation time; calling a task could suspend the process. Yes, procedural functions can have output arguments.

9. **Question 9 (Pass by Reference):** What error occurs if a module-level task declares `ref int a` without the `automatic` keyword?

Answer: Compiler error. IEEE 1800-2017 Clause 13.5.2 requires automatic lifetime for `ref` formals in static module scopes.

10. **Question 10 (Passing Handles):** Inside a task with input `Packet pkt`, code executes `pkt.len = 100;` `pkt = null;`. What happens to the caller's object and handle?

Answer: The caller's object property `len` is updated to 100. The caller's handle **remains valid and non-null** (only the local formal was nulled).

11. **Question 11 (Constructor Syntax):** Why does function `new()` have no return type declared?

Answer: The constructor implicitly allocates and returns an object handle of its class type. Declaring any return type is a syntax error.

12. **Question 12 (Shallow Copy):** If class `Outer` has handle `Inner in_inst;`, what is shared after `o2 = new o1;`? Does `new o1` execute `Outer`'s constructor?

Answer: `o1` and `o2` share the exact same `in_inst` object on the heap. **No**, shallow copy does NOT run the constructor or field initializers.

13. **Question 13 (Deep Copy):** How does deep copy differ from shallow copy?

Answer: Deep copy recursively instantiates and copies every nested child object (`copy.in_inst = in_inst.copy()`), achieving complete memory independence.

14. **Question 14 (Virtual Methods):** Base `b`; Child `c = new(); b = c; b.display();`. If `display()` is virtual in `Base`, which method runs?

Answer: `Child.display()` executes via dynamic dispatch based on the runtime object type.

15. **Question 15 (Super Keyword):** Where must `super.new(...)` be placed in a derived class constructor? What happens if omitted?

Answer: It must be the **first executable statement**. If omitted, an implicit `super.new()` is called, which fails if the base constructor requires arguments.

16. **Question 16 (Randomization Assertions):** Why is calling `trans.randomize()` without `assert` considered a critical defect in testbenches?

Answer: If the constraint solver fails, unrandomized stale data is driven into the DUT silently, masking hardware defects. Wrapping in `assert(trans.randomize()) else $fatal` guarantees immediate failure visibility.

Part VI: Simulation Verification Report & Standards Appendix

Simulator Verification Summary

The technical claims, code corrections, and runtime traces in this handbook were compiled, elaborated, and simulated using **AMD Vivado Simulator 2024.1 (64-bit, build 5076996)** with standard-compliance flags `xvlog -sv`, `xelab -s`, and `xsim -runall`.

```
=====
SIMULATION VERIFICATION RESULTS -- AMD Vivado Simulator 2024.1
Testbench: revision_checks.sv | Resolution: 1 ps | Finish Time: 1234 ps
=====
TRACE types bit[7:0]=255 byte=-1 defaults(bit,logic)=0,x
TRACE time $time=1.000 ns $realtime=1.234 ns
TRACE arrays concat='{1'b1,1'b0,1'b1,1'b1}' resized='{11,22,33,0,0}' queue='{1,8,3,4}'
TRACE arguments sum=12 input(actual,result)=10,15 swap=9,4 ctor_out=26
TRACE copies source=10/88 shallow=99/88 deep=123/456 ctor_calls=2
TRACE dispatch virtual=3 nonvirtual=20
TRACE randomization solver checks passed: 10 transactions generated with range exclusions
=====
STATUS: PASS -- 32 of 32 deterministic checks completed with zero errors.
=====
```

Primary Language Standards References (IEEE Std 1800-2017)

- **Clause 6 (Data Types):** §6.8 (2-state/4-state), §6.11 (Signedness & truncation), Table 6-7 & 6-8 (Defaults).
- **Clause 7 (Arrays & Queues):** §7.4 (Packed/Unpacked), §7.5 (Dynamic arrays & `new[N]`), §7.10 (Queues & methods).
- **Clause 8 (Classes & OOP):** §8.5 (Handles vs Objects), §8.7 (Constructors), §8.12 (Shallow copy), §8.20 (Polymorphism & virtual methods), §8.22 (Upcasting/Downcasting).
- **Clause 10 (Assignments):** §10.9 (Assignment patterns), §10.10 (Unpacked concatenation).
- **Clause 13 (Tasks & Functions):** §13.3 (Tasks), §13.4 (Functions & outputs), §13.5 (Argument passing & ref).
- **Clause 18 (Constrained Randomization):** §18.4 (Random variables `rand/randc`), §18.5 (Constraint blocks & inside operator), §18.11 (In-line `randomize()` checking).
- **Clause 20 (Utility System Functions):** §20.3 (`$time`, `$realtime`), §20.4 (`$timeformat`), §20.7 (`$size`, `$bits`).